

Modelagem formal do sistema

Smart Tennis Arena — alvo interativo TOBIAS

Estados, eventos, linguagem, expressão regular e gramática da sessão de treino

Teoria da Computação e Linguagens Formais

Entrega 1

Integrantes

Gabriel Carvalho Mota — 24025708

Guilherme de Lima Siqueira — 21010649

Rodrigo Luiz Menezes dos Reis — 24036593

Vitória Leticia Maciel da Silva — 24026593

Sumário

Contexto do Projeto.....	3
1. Estados do sistema	3
Sobre a calibração (estado S2)	3
2. Eventos (entradas)	4
3. Diagrama de estados	4
4. Tabela de transições	5
5. Linguagem do sistema	5
Sequências válidas	6
Sequências inválidas	6
6. Expressão regular.....	6
Como eu validei	6
7. Gramática	8
8. Justificativa da modelagem	8
Por que esses estados.....	8
Onde cada parte aparece no sistema.....	8
O que eu simplifiquei.....	9
Por que um autômato finito é suficiente	9
Minimização	9
Uma garantia que dá para provar no modelo	10

Contexto do Projeto

O nosso projeto é um alvo interativo em forma de boneco João-bobo, nomeado TOBIAS. O praticante acerta o alvo com a bola de tênis, um sensor de aceleração dentro do boneco detecta a batida, o próprio dispositivo reage na hora com som e luz, e manda o evento por Bluetooth para o aplicativo, que conta os pontos e manipula os dados necessários.

Nós modelamos o ciclo de vida de uma sessão de treino: a sequência de comandos e eventos desde o momento em que o celular se conecta ao alvo até o fim da sessão. Ela é o núcleo do sistema, aparece tanto no firmware do ESP32 quanto no aplicativo, e cada passo dela corresponde a alguma coisa que dá para observar e registrar: uma mensagem Bluetooth, uma leitura do sensor ou um toque na tela.

1. Estados do sistema

O sistema tem seis estados nomeados, mais um estado de erro. O estado inicial é S0 e o único estado final é S5, que representa a sessão encerrada.

Código	Nome	O que significa	Final
S0	Desconectado	O alvo está ligado e se anunciando por Bluetooth, mas nenhum celular está conectado. Nenhum comando de treino é aceito aqui.	não
S1	Conectado	O aplicativo pareou com o alvo e já consegue ver bateria, versão do firmware e status do sensor. Ainda não dá para começar um treino.	não
S2	Calibrado	O dispositivo tem uma referência de força válida e está pronto para executar qualquer treino do catálogo.	não
S3	Em execução	A sessão está rodando: o alvo emite o sinal de chamada, lê o sensor, detecta as batidas, toca o áudio e acende os LEDs. É o único estado em que acontecem acertos e erros.	não
S4	Pausada	O treino foi pausado. O som e os LEDs param e o sensor deixa de gerar eventos, mas a pontuação e a sequência de acertos continuam guardadas.	não
S5*	Encerrada	A sessão terminou. O resumo é fechado e os eventos ficam prontos para sincronizar com o servidor.	sim
SE	Erro	Estado de rejeição. Recebe qualquer comando que não faz sentido no momento em que chegou. O comando é descartado e registrado no log, e a cadeia nunca é aceita. Não desenhei ele no diagrama para não poluir.	não

Sobre a calibração (estado S2)

A calibração é o que dá ao sistema uma referência de força. Nós determinamos, com base no cálculo feito pelo sensor uma faixa média de referência para determinarmos a relevância de força e pontuação do lance do jogador. A partir daí, cada batida do treino vale em relação a essa referência, e não em valor bruto. Isso é necessário porque a leitura do sensor muda conforme o quanto o boneco está cheio, a altura em que o sensor está colado e a firmeza da fixação. Sem a referência, o mesmo golpe daria pontuações diferentes em montagens diferentes.

Um ponto importante para entender o modelo: essa calibragem não acontece toda vez que alguém vai treinar. A referência fica gravada na memória do ESP32, que não apaga quando o aparelho desliga. Ela só precisa ser refeita quando a montagem física muda. Na prática, vamos calibrar duas vezes: no teste anterior

à apresentação e de novo no dia da apresentação, porque o boneco vai ser transportado e o sensor pode sair do lugar.

Mesmo assim manteve o estado S2 no modelo, porque o sistema sempre passa por ele antes de iniciar um treino. O que muda é o custo da transição: quando já existe referência salva, o aplicativo apenas carrega o valor e segue.

2. Eventos (entradas)

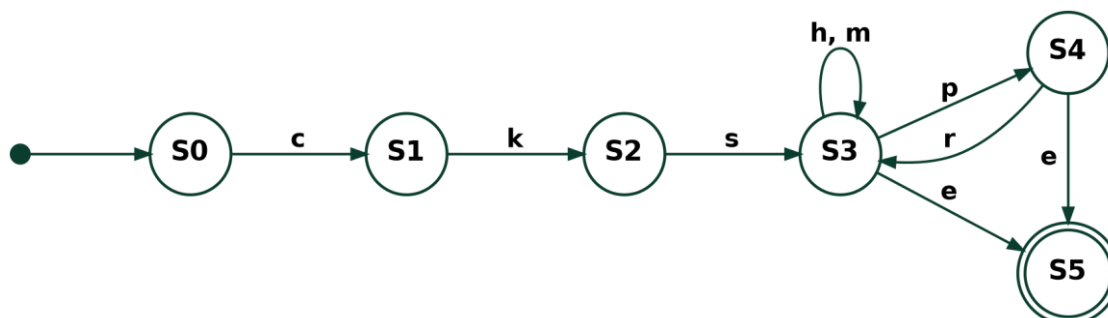
São oito eventos. Alguns são comandos do usuário, que chegam pelo aplicativo, outros são gerados pelo próprio dispositivo a partir do sensor e dos temporizadores. Para a máquina de estados, os dois tipos são apenas símbolos de entrada.

$$\Sigma = \{ c, k, s, h, m, p, r, e \}$$

Sím.	Evento	Quem gera	O que acontece
c	conectar	Aplicativo	O celular parecia com o alvo por Bluetooth e recebe o status do dispositivo.
k	calibrar	Aplicativo	Carrega a referência.
s	iniciar	Aplicativo	Carrega os parâmetros do treino e emite o primeiro sinal de chamada.
h	acerto	Dispositivo (sensor)	Pico de aceleração acima do limiar. Toca o áudio, acende o LED, avisa o aplicativo e soma pontos.
m	erro ou tempo esgotado	Dispositivo (temporizador)	O tempo do sinal de chamada acabou sem batida. Aplica a penalidade e zera a sequência.
p	pausar	Aplicativo	Silencia os atuadores e para de gerar eventos, guardando o contexto.
r	retomar	Aplicativo	Volta a emitir o sinal de chamada e a ler o sensor.
e	encerrar	Aplicativo, ou o próprio treino quando os lances acabam	Fecha a sessão e monta o resumo.

3. Diagrama de estados

O diagrama abaixo mostra a máquina de estados. O estado de erro SE não aparece: toda letra que não tem seta em um estado leva para ele.



Máquina de estados da sessão de treino. S0 desconectado, S1 conectado, S2 calibrado, S3 em execução, S4 pausada, S5 encerrada (final).

Duas coisas do desenho importantes. O laço em S3 existe porque um acerto ou um erro não mudam a situação do sistema: a sessão continua rodando, e quem conta os pontos é a pontuação, não a máquina de estados. E o par S3 e S4 só fecha com pausar e retomar juntos, o que impede duas pausas seguidas ou uma retomada sem pausa.

4. Tabela de transições

Cada linha corresponde a uma seta. Incluí também as linhas do estado de erro, que no diagrama ficam implícitas.

Estado atual	Evento	Próximo estado	Ação
S0	c	S1	Conecta e envia o status do dispositivo
S1	k	S2	Carrega ou estabelece a referência de força
S2	s	S3	Carrega o treino e emite o sinal de chamada
S3	h	S3	Toca o áudio, acende o LED, registra o acerto
S3	m	S3	Registra o erro e zera a sequência
S3	p	S4	Silencia os atuadores e guarda o contexto
S4	r	S3	Reemite o sinal de chamada
S3	e	S5	Fecha a sessão e monta o resumo
S4	e	S5	Fecha a sessão que estava pausada
S0, S1, S2	qualquer outro símbolo	SE	Descarta o comando e registra a tentativa no log
S3, S4	qualquer outro símbolo	SE	Descarta o comando sem interromper a sessão
S5	qualquer símbolo	SE	A sessão já acabou; outra exige uma nova cadeia
SE	qualquer símbolo	SE	Permanece no erro e nunca aceita

Como para cada par de estado e símbolo existe um único destino, a máquina é determinística. No firmware, essa tabela vira uma única função de transição consultada no laço principal.

5. Linguagem do sistema

A linguagem do sistema é o conjunto das cadeias que descrevem uma sessão de treino completa, do pareamento até o encerramento:

$$L = \{w \in \Sigma^* \mid \text{começando em } S0 \text{ e lendo } w, \text{ o autômato termina em } S5\}$$

Em palavras: toda sessão válida começa conectando, calibrando e iniciando, aceita qualquer quantidade de acertos, erros e pares de pausa e retomada, e termina com o encerramento. Uma sessão que ainda está em andamento não pertence à linguagem, porque não foi fechada.

Sequências válidas

Cadeia	O que aconteceu
ckse	Conectou, calibrou, iniciou e encerrou sem nenhum lance.
ckshhe	Três acertos seguidos e encerramento normal.
cksmhme	Um erro, um acerto, outro erro e encerramento.
ckshprhe	Acertou, pausou para juntar as bolas, retomou, acertou e encerrou.
ckshprprhhe	Duas pausas completas ao longo da sessão.
ckspe	Pausou e encerrou direto, sem retomar.

Sequências inválidas

Cadeia	Por que é rejeitada
cshhe	Tentou iniciar sem calibrar.
kcshe	Tentou calibrar antes de conectar.
cksh	A sessão começou e nunca foi encerrada.
ckshrhe	Retomou sem ter pausado.
cksppe	Duas pausas seguidas, sem retomada entre elas.
ckshep	Mandou comando depois que a sessão já tinha acabado.

6. Expressão regular

A mesma linguagem descrita como expressão regular:

$$R = cks(h|m|pr)^*(e|pe)$$

Em palavras: cks é o começo obrigatório, conectar, calibrar e iniciar; o bloco do meio é o treino em si, com qualquer quantidade de acertos, erros e pares de pausa e retomada, e o final é o encerramento, que pode acontecer com a sessão rodando ou pausada. O par pr aparece grudado de propósito, porque é isso que impede pausa sem retomada no meio do treino.

O JFLAP não aceita a barra vertical: ele usa o sinal de mais para união. Então, para testarmos na ferramenta, a expressão foi digitada assim:

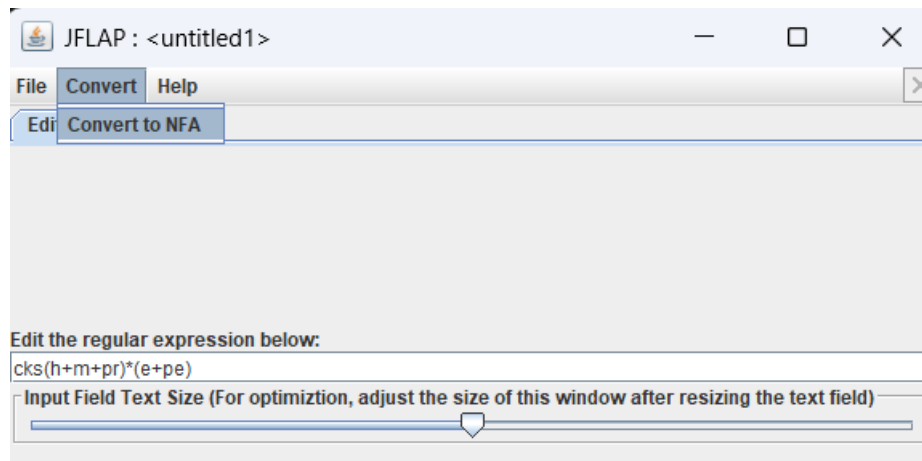
$$cks(h+m+pr)^*(e+pe)$$

Como eu validei

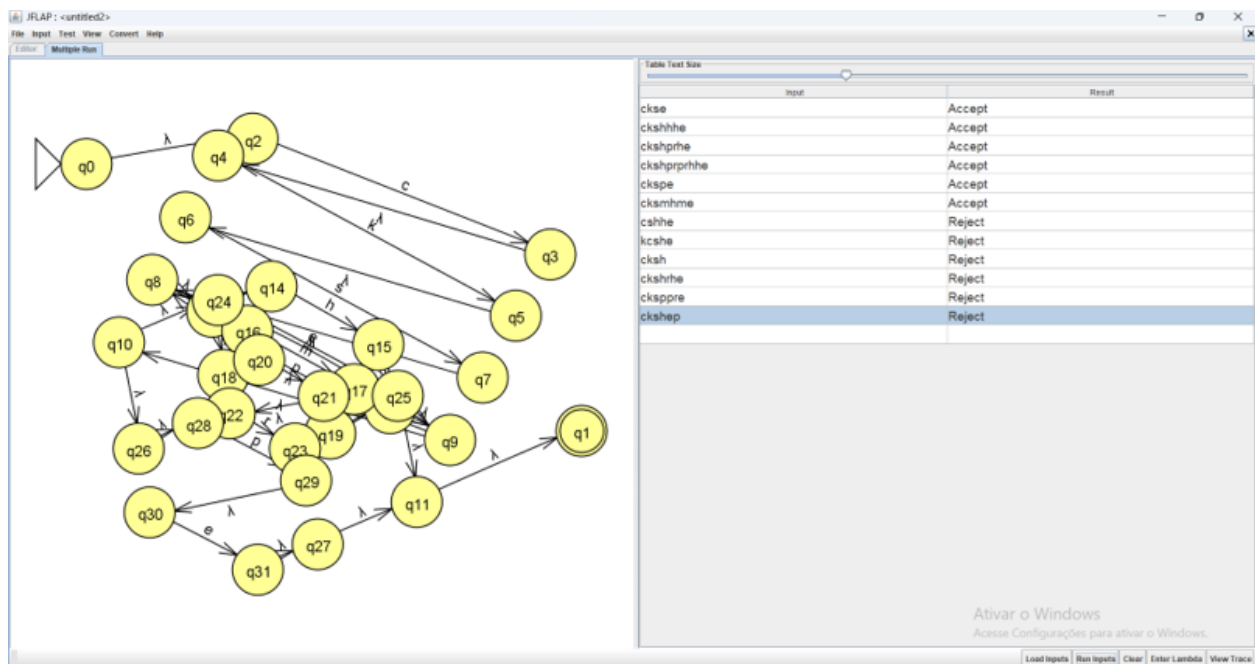
Testei de duas formas. Primeiro escrevi um script que simula a tabela de transições e compara o resultado com o da expressão regular para todas as cadeias possíveis até seis símbolos, e também para essas cadeias precedidas dos prefixos c, ck e cks. Os dois concordaram em todos os casos. Depois testei no JFLAP com a lista abaixo.

Cadeia	Esperado	Motivo
ckse	Aceita	Sessão vazia, mas completa

Cadeia	Esperado	Motivo
ckshhhe	Aceita	Três voltas no laço de S3
ckshprhe	Aceita	Ciclo de pausa e retomada fechado
ckshprprhhe	Aceita	Dois ciclos de pausa
ckspe	Aceita	Encerramento a partir da pausa
cksmhme	Aceita	Erros intercalados com acerto
cshhe	Rejeita	Sem calibração
kcshe	Rejeita	Ordem invertida no começo
cksh	Rejeita	Termina em S3, que não é final
ckshrhe	Rejeita	Retomada sem pausa
cksppe	Rejeita	Pausas consecutivas
ckshep	Rejeita	Comando depois do encerramento



Expressão regular digitada no JFLAP.



Resultado do teste: as seis primeiras cadeias aceitas e as seis últimas rejeitadas, como previsto na tabela acima.

7. Gramática

Montei a gramática a partir do autômato, seguindo o método visto em aula: cada estado virou uma variável e cada transição virou uma produção. S corresponde a S0, A a S1, B a S2, C a S3, D a S4 e F a S5.

$V = \{ S, A, B, C, D, F \}$ $T = \{ c, k, s, h, m, p, r, e \}$
Símbolo inicial: S

$S \rightarrow c A$
 $A \rightarrow k B$
 $B \rightarrow s C$
 $C \rightarrow h C \mid m C \mid p D \mid e F$
 $D \rightarrow r C \mid e F$
 $F \rightarrow \epsilon$

Conferindo o formato de cada produção: todas são um terminal seguido de uma variável, ou só a cadeia vazia. Nenhuma tem variável no meio nem duas variáveis do mesmo lado. Então é uma gramática regular linear à direita, que é coerente com a linguagem ser reconhecida por um autômato finito.

Tirando a produção vazia, ela fica mais curta e gera exatamente a mesma linguagem:

$S \rightarrow c A$ $A \rightarrow k B$ $B \rightarrow s C$
 $C \rightarrow h C \mid m C \mid p D \mid e$
 $D \rightarrow r C \mid e$

Derivando a cadeia ckshprhe, que é acerto, pausa, retomada, acerto e encerramento:

$S \Rightarrow cA \Rightarrow ckB \Rightarrow cksC \Rightarrow ckshC \Rightarrow ckshpD$
 $\Rightarrow ckshprC \Rightarrow ckshprhC \Rightarrow ckshprhe$

E tentando derivar uma cadeia inválida, ckshrhe: depois de chegar em C, a única forma de usar o r seria estando em D, e só se chega em D pelo p. Como não teve pausa, não existe derivação possível, e a cadeia não pertence à linguagem.

8. Justificativa da modelagem

Por que esses estados

Para desenhar a máquina, eu me perguntei o que o sistema precisa lembrar para decidir se um comando pode ser aceito. As respostas foram: já tem celular conectado, já tem referência de força, o treino está rodando, o treino está pausado, o treino acabou. Cada resposta diferente virou um estado. Por isso não existe um estado para cada lance do treino: quantos acertos já aconteceram não muda em nada o que o sistema pode aceitar a seguir.

Onde cada parte aparece no sistema

Elemento formal	Onde está na implementação
Estados S0 a S5	Um enum no firmware em C++ e outro igual no aplicativo em Flutter
Símbolos do alfabeto	Comandos enviados pelo aplicativo por Bluetooth e eventos gerados pelo sensor
Função de transição	Uma única função consultada no laço principal do firmware
Estado SE	O caso padrão dessa função: comando descartado e registrado no log
Linguagem L	CrITÉrio para validar se um log de sessão está bem formado

Elemento formal	Onde está na implementação
Expressão regular	Base dos testes automáticos de sequência

O que eu simplifiquei

Duas simplificações conscientes. A primeira: por dentro, o firmware divide o estado S3 em três fases, esperando a batida, capturando a janela de medição e cumprindo o período em que ignora novas batidas para o balanço do boneco não contar várias vezes. Essas fases mudam sozinhas por tempo, sem consumir nenhum símbolo do alfabeto, então juntei tudo em um estado só. A segunda: os valores numéricos da batida, como o pico de aceleração e o tempo de reação, viajam junto com o evento, mas não entram no alfabeto. Se entrassem, o alfabeto seria infinito e a linguagem deixaria de ser regular, sem nenhum ganho, porque o controle não decide nada com base no valor exato.

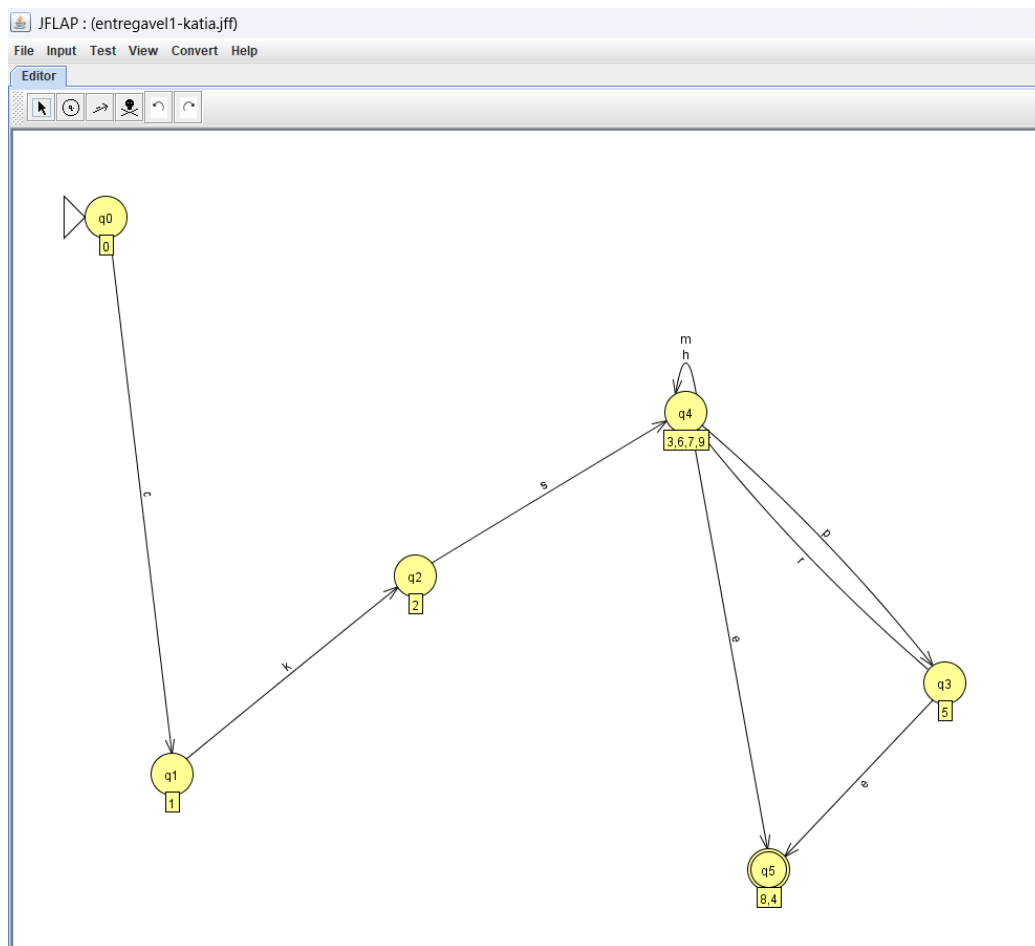
Por que um autômato finito é suficiente

O sistema tem um número fixo de estados e nenhuma decisão dele precisa de memória ilimitada. O ponto mais delicado é a sequência de acertos, que aumenta o multiplicador da pontuação. Como vamos determinar um limite para esse multiplicador, por decisão nossa de design do jogo, esse contador ainda caberia no conjunto de estados. Se ele crescesse sem limite, aí seria preciso contar indefinidamente e o modelo adequado passaria a ser um autômato com pilha. Ou seja, foi a regra do jogo que definiu a classe do modelo. Como decidimos tratar a pontuação como uma conta feita sobre os eventos, e não como parte do controle, o autômato finito dá conta.

Minimização

Apliquei o método de particionamento visto em aula: separei os estados finais dos não finais e fui refinando enquanto houvesse, dentro de um mesmo grupo, estados cujas transições levassem a grupos diferentes. Cada estado ficou sozinho na sua classe, inclusive S3 e S4, que se distinguem porque o acerto mantém a aceitação possível a partir de S3 e a destrói a partir de S4. Nenhum par de estados é equivalente, então o autômato que eu desenhei já é o mínimo.

Confirmamos esse resultado no JFLAP. Partindo da expressão regular, a ferramenta gerou um autômato não determinístico de 32 estados, converteu para determinístico e aplicou a minimização. O autômato mínimo obtido tem a mesma estrutura do que desenhamos a partir do funcionamento do sistema: seis estados, o laço dos acertos e erros, o ciclo de pausa e retomada e um único estado final. Ou seja, dois caminhos independentes chegaram ao mesmo desenho.



Autômato mínimo gerado pelo JFLAP a partir da expressão regular. Os estados q0 a q5 correspondem a S0, S1, S2, S4, S3 e S5.

Uma garantia que dá para provar no modelo

Uma regra importante do sistema é que nenhuma batida pode ser pontuada antes de o alvo estar calibrado, senão a pontuação não seria comparável. Isso deixa de ser uma promessa e vira uma propriedade verificável nas três representações. No autômato, só se chega em S3 passando por S1 e S2. Na expressão regular, nenhum h aparece antes do bloco cks. Na gramática, a variável C, que é a única que produz h, só é alcançada depois de c, k e s.